

JAVA 17 OCP · YDS STUDY SERIES

Unit 15 · JDBC — Technical Memory Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu not, [ana çift dilli dersin](#) Java 17 JDBC kurallarını OCP sınavında hızlı karar vermeyi kolaylaştıracak biçimde özetler. Buradaki mini quiz özgün çalışma sorularından oluşur; gerçek sınav sorusu değildir.

[Vocabulary](#) · [Grammar notes](#)

1. İlişkisel veritabanı ve CRUD

Relational database (ilişkisel veritabanı), veriyi satır ve sütunlardan oluşan tablolar halinde saklar.

İşlem	SQL keyword	Temel sonuç
Create	INSERT	Yeni row ekler
Read	SELECT	Row/column verisi okur
Update	UPDATE	Sıfır veya daha fazla row değiştirir
Delete	DELETE	Sıfır veya daha fazla row siler

SQL keyword'leri case-insensitive'dir. OCP sorularında SQL'in doğru olduğu, aksi açıkça belirtilmedikçe tablo ve column adlarının geçerli olduğu varsayılır.

Memory tip: CRUD harfleri SQL keyword'lerinin ilk harfleriyle tam eşleşmez: Create işlemi **INSERT**, Read işlemi **SELECT** kullanır.

2. Beş temel JDBC interface'i

Interface	Görevi
Driver	JDBC URL'yi anlayan database driver'ını temsil eder
Connection	Database session ve transaction context'ini temsil eder
PreparedStatement	Parametrelili veya parametresiz SQL'i yürütür
CallableStatement	Stored procedure çağırır
ResultSet	Query sonucundaki row/column verisini taşır

Interface'ler JDK'deki `java.sql` package'ındadır. Concrete implementation'lar database'e özgü driver JAR'ından gelir. `DriverManager` ise interface değil, JDK'nin sağladığı concrete factory class'tır.

Modüler projede:

```
module zoo.app {
    requires java.sql;
}
```

3. JDBC URL

Genel form:

```
jdbc:<subprotocol>:<subname>
```

Örnekler:

```
jdbc:hsqldb:file:zoo
jdbc:postgresql://localhost:5432/zoo
jdbc:oracle:thin:@192.0.2.10:1521:zoo
```

- İlk parça her zaman `jdbc` protocol'üdür.
- İkinci parça product/vendor'a özgü subprotocol'dür.
- Subname tamamen driver'a özgüdür; host, port, database veya file bilgisi içerebilir.

OCP trap: Ortak parçaları colon (:) ayırır. URL'nin bütün ayrıntıları için tek bir vendor-independent format yoktur.

4. Connection edinme

```
import java.sql.DriverManager;
import java.sql.SQLException;

class Connect {
    static void open(String url) throws SQLException {
        try (var conn = DriverManager.getConnection(url)) {
            System.out.println(conn.getClass().getName());
        }
    }
}
```

İki temel overload:

```
DriverManager.getConnection(url);
DriverManager.getConnection(url, username, password);
```

Return type `Connection`dir; çalışma zamanındaki concrete type driver'a aittir. Uygun driver bulunamazsa veya bağlantı kurulamazsa `SQLException` oluşur.

Modern JDBC driver'larında service-provider mekanizması kullanıldığından normal durumda `Class.forName()` ile driver yüklemek gerekmez.

5. PreparedStatement oluşturma

SQL, statement oluşturulurken verilmelidir:

```
String sql = "SELECT id, name FROM exhibits";
try (var ps = conn.prepareStatement(sql)) {
    // execute later
}
```

Bu çağrı yoktur:

```
conn.prepareStatement(); // DOES NOT COMPILE
```

Neden: `Connection.prepareStatement()` için no-argument overload bulunmaz. SQL daha sonra `executeQuery(sql)` gibi bir çağrıyla verilmez; SQL zaten `PreparedStatement` içinde tutulur.

6. Doğru execute method'unu seçme

Method	Normal kullanım	Return type
<code>executeQuery()</code>	<code>SELECT</code>	<code>ResultSet</code>
<code>executeUpdate()</code>	<code>INSERT, UPDATE, DELETE</code>	<code>int</code> affected-row count
<code>execute()</code>	Query veya update bilinmiyorsa	<code>boolean</code>

```
try (var ps = conn.prepareStatement(
    "UPDATE exhibits SET num_acres = num_acres + 1")) {
    int changed = ps.executeUpdate();
}
```

`execute()` sonucu:

- `true`: İlk sonuç `ResultSet` tir.
- `false`: İlk sonuç update count'tur veya sonuç yoktur.

```
boolean queryResult = ps.execute();
if (queryResult) {
    try (ResultSet rs = ps.getResultSet()) {
        // read rows
    }
} else {
    int count = ps.getUpdateCount();
}
```

OCF trap: Boolean “statement başarılı mı?” anlamına gelmez. Sonucun `ResultSet` olup olmadığını gösterir.

Yanlış `execute` method'u Java compiler tarafından SQL string'ine bakılarak yakalanamaz. Kod derlenir; driver runtime'da `SQLException` oluşturabilir.

7. Bind variable

Question mark (?) bir bind variable'dır:

```
String sql = "INSERT INTO names VALUES(?, ?, ?)";
try (var ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 6);
    ps.setInt(2, 1);
    ps.setString(3, "Edith");
    ps.executeUpdate();
}
```

Kurallar:

- Parameter index 1 ile başlar.
- Setter'lar herhangi bir sırada çağrılabilir.
- Her placeholder execution öncesinde değer almalıdır.
- Olmayan index'i set etmek veya eksik parameter ile execute etmek `SQLException` oluşturur.
- Java compiler SQL içindeki placeholder sayısını kontrol etmez.

Memory tip: JDBC'de bind parameter ve result column index'i 1 tabanlıdır.

8. Sık kullanılan setter'lar

Method	Java parameter type
<code>setBoolean(int, boolean)</code>	<code>boolean</code>
<code>setDouble(int, double)</code>	<code>double</code>
<code>setInt(int, int)</code>	<code>int</code>
<code>setLong(int, long)</code>	<code>long</code>
<code>setObject(int, Object)</code>	<code>Object</code>
<code>setString(int, String)</code>	<code>String</code>

`setNull()` özel durumdur:

```
ps.setNull(2, java.sql.Types.VARCHAR);
```

İkinci argument Java value değil, target SQL type'ını bildiren `int` constant'tır.

`setObject()` primitive argument'i autoboxing ile kabul edebilir:

```
ps.setObject(1, 42); // Integer olarak geçirilir
```

9. Statement'ı yeniden kullanmak

Set edilen parameter value'ları statement üzerinde tutulur:

```
String sql = "SELECT id FROM exhibits WHERE num_acres > ?";
try (var ps = conn.prepareStatement(sql)) {
    ps.setDouble(1, 3.0);
    try (var first = ps.executeQuery()) {
        // first query
    }

    ps.setDouble(1, 100.0);
    try (var second = ps.executeQuery()) {
        // second query
    }
}
```

İkinci execution öncesinde parameter değiştirilmezse önceki value yeniden kullanılır.

`clearParameters()` value'ları temizler; sonra gerekli parameter'lar yeniden set edilmeden execute edilirse `SQLException` beklenir.

10. Batch update

```
String sql = "INSERT INTO names VALUES(?, ?, ?)";
try (var ps = conn.prepareStatement(sql)) {
    for (int id = 10; id < 12; id++) {
        ps.setInt(1, id);
        ps.setInt(2, 1);
        ps.setString(3, "Elephant-" + id);
        ps.addBatch();
    }
    int[] counts = ps.executeBatch();
}
```

`addBatch()` current parameter set'ini batch'e ekler. `executeBatch()` bir `int[]` döndürür. Her element driver'ın ilgili command için bildirdiği update sonucudur; her database aynı batching optimizasyonunu sunmak zorunda değildir.

11. ResultSet cursor modeli

Yeni `ResultSet` cursor'ı ilk row'un önündedir:

```
before first → row 1 → row 2 → ... → after last
                rs.next()
```

```
try (var ps = conn.prepareStatement("SELECT id, name FROM exhibits");
    var rs = ps.executeQuery()) {
    while (rs.next()) {
        int id = rs.getInt("id");
        String name = rs.getString("name");
    }
}
```

- `next()` cursor'ı ilerletir.
- Valid row'a ulaştığında `true`, row kalmadığında `false` döndürür.
- `next()` öncesi veya `false` sonrasında getter çağrısı `SQLException` oluşturur.
- Tek row bekleniyorsa `if (rs.next())`, çok row için `while (rs.next())` kullanılır.

12. Column'a erişme

İsimle:

```
int id = rs.getInt("id");
String name = rs.getString("name");
```

Index ile:

```
int id = rs.getInt(1);
String name = rs.getString(2);
```

Column index'i 1 ile başlar. Olmayan name veya index runtime `SQLException` üretir.

Sık getter'lar:

Method	Return type
<code>getBoolean()</code>	<code>boolean</code>
<code>getDouble()</code>	<code>double</code>
<code>getInt()</code>	<code>int</code>
<code>getLong()</code>	<code>long</code>
<code>getObject()</code>	<code>Object</code>
<code>getString()</code>	<code>String</code>

Her biri column name veya `int` index alabilen overload'lara sahiptir.

13. SQL NULL ayrıntısı

Primitive getter SQL `NULL` için primitive default value döndürebilir:

```
int value = rs.getInt("amount");
if (rs.isNull()) {
    // SQL NULL idi; gerçek 0 değeri değildi
}
```

Bu ayrıntı, 0 ile SQL `NULL` ayırmak gerektiğinde önemlidir. `getObject()` uygun bir reference type veya `null` ile çalışabilir.

14. getObject() ve pattern matching

```
Object field = rs.getObject("id");
if (field instanceof Integer id) {
    System.out.println(id);
}
```

Concrete Java type, SQL type'a ve driver mapping'ine bağlıdır. Assignment conversion yine normal Java kurallarına uyar:

```
String value = rs.getObject(1); // DOES NOT COMPILE
```

Çünkü `getObject(int)` static olarak `Object` döndürür.

15. CallableStatement

`CallableStatement`, stored procedure çağırmak için `Connection.prepareCall()` ile oluşturulur:

```
String sql = "{call read_e_names()}";
try (var cs = conn.prepareCall(sql);
    var rs = cs.executeQuery()) {
    while (rs.next()) {
        System.out.println(rs.getString(3));
    }
}
```

Exam syntax'inde braces (`{}`), `call` keyword'ü ve her parameter için `?` beklenir. Bazı driver'ların daha esnek davranması portable JDBC kuralını değiştirmez.

`prepareStatement()` SQL statement; `prepareCall()` stored procedure call içindir. Her ikisi de compile-time'da SQL semantiğini doğrulamaz.

16. IN, OUT ve INOUT

Kural	IN	OUT	INOUT
Input taşır	Evet	Hayır	Evet
Output taşır	Hayır	Evet	Evet
Setter gerekir	Evet	Hayır	Evet
<code>registerOutParameter()</code> gerekir	Hayır	Evet	Evet

IN:

```
String sql = "{call read_names_by_letter(?)}";
try (var cs = conn.prepareCall(sql)) {
    cs.setString(1, "Z");
    try (var rs = cs.executeQuery()) {
        // rows
    }
}
```

OUT:

```
String sql = "{call magic_number()}";
try (var cs = conn.prepareCall(sql)) {
    cs.registerOutParameter(1, java.sql.Types.INTEGER);
    cs.execute();
    int answer = cs.getInt(1);
}
```

INOUT:

```
String sql = "{call double_number()}";
try (var cs = conn.prepareCall(sql)) {
    cs.setInt(1, 8);
    cs.registerOutParameter(1, java.sql.Types.INTEGER);
    cs.execute();
    int doubled = cs.getInt(1);
}
```

OCP trap: `INOUT` için hem setter hem `registerOutParameter()` gerekir.

17. Parameter name veya index

`CallableStatement` parameter'ları driver/stored-procedure desteğine göre name veya index ile kullanabilir:

```
cs.setString(1, "Z");
cs.setString("prefix", "Z");
```

Kaynak exam sorularında ikisi eşdeğer kabul edilir. Index kullanıldığında sayaç yine 1'den başlar.

18. `ResultSet` type ve concurrency seçenekleri

Statement oluştururken ek `int` constant'lar geçirilebilir:

```
var ps = conn.prepareStatement(
    sql,
    ResultSet.TYPE_FORWARD_ONLY,
    ResultSet.CONCUR_READ_ONLY
);
```

Type:

- `TYPE_FORWARD_ONLY`
- `TYPE_SCROLL_INSENSITIVE`
- `TYPE_SCROLL_SENSITIVE`

Concurrency:

- `CONCUR_READ_ONLY`
- `CONCUR_UPDATABLE`

Her driver bütün kombinasyonları desteklemek zorunda değildir. Bunlar enum değil, `int` constant'lardır. İki optional argument'in sırası type, ardından concurrency'dir.

19. Autocommit ve transaction

Yeni `Connection` için normal default autocommit `true`dur:

```
conn.setAutoCommit(false);
try {
    // related updates
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
    throw e;
}
```

Method	Etki
<code>setAutoCommit(false)</code>	Manual transaction control başlatır
<code>commit()</code>	Current transaction değişikliklerini kalıcılaştırır
<code>rollback()</code>	Current transaction değişikliklerini geri alır
<code>setAutoCommit(true)</code>	Mode'u açar; active transaction varsa commit eder

Autocommit `true` iken her statement kendi transaction'ında otomatik commit edilir; daha sonra yapılan `rollback()` bu committed değişiklikleri geri almaz.

Contract notu: Manual transaction açıkken `Connection.close()` çağrısında commit/rollback sonucuna güvenilmemelidir; JDBC contract sonucu implementation-defined bırakır. Explicit `commit()` veya `rollback()` yap.

20. Transaction başarı kontrolü

Birden fazla update tek logical işlemse autocommit kapatılır:

```
conn.setAutoCommit(false);
try {
    boolean first = updateFirst(conn);
    boolean second = updateSecond(conn);
    if (first && second) {
        conn.commit();
    } else {
        conn.rollback();
    }
} catch (SQLException e) {
    conn.rollback();
    throw e;
}
```

`executeUpdate()` affected-row count'ını kontrol etmek, beklenen row gerçekten değişmediyse transaction'ı geri alma kararı vermeyi sağlar.

21. Savepoint

```
conn.setAutoCommit(false);
Savepoint beforeDetails = conn.setSavepoint();
// changes A
Savepoint beforeOptional = conn.setSavepoint("optional");
// changes B
conn.rollback(beforeOptional); // B gider; A kalır
conn.commit();
```

- `setSavepoint()` unnamed bookmark oluşturur.
- `setSavepoint(String)` named bookmark oluşturur.
- `rollback(savepoint)` yalnız o noktadan sonraki değişiklikleri geri alır.
- Daha önceki bir savepoint'e rollback, ondan sonra oluşturulan savepoint'leri geçersiz kılar.

- Full `rollback()` bütün current transaction değişikliklerini ve savepoint state'ini geçersiz kılar.
- Target savepoint rollback nedeniyle otomatik olarak release edilmez. Hâlâ valid olan aynı savepoint'e yeniden `rollback(savepoint)` çağrısı JDBC contract tarafından yasaklanmaz.

Geçersiz savepoint'e rollback runtime `SQLException` üretir.

Kaynak errata: Review Question 15 Appendix anahtarı C, D'dir; fakat `rollback(curly)` seçeneği target `curly`yi release etmediğinden B de portable JDBC değerlendirmesinde geçerlidir. Savunulabilir küme B, C, D'dir.

22. JDBC resource sırası

Oluşturma:

```
Connection → PreparedStatement/CallableStatement → ResultSet
```

Kapatma:

```
ResultSet → PreparedStatement/CallableStatement → Connection
```

Try-with-resources reverse declaration order'da kapattığından doğal çözüm:

```
try (var conn = DriverManager.getConnection(url);
    var ps = conn.prepareStatement(sql);
    var rs = ps.executeQuery()) {
    while (rs.next()) {
        System.out.println(rs.getString(1));
    }
}
```

`Connection.close()` onun oluşturduğu statement/result resource'larını; statement'ın kapanması onun current `ResultSet`ini kapatır. Yine de ownership'i açık gösteren try-with-resources en güvenli ve okunabilir biçimdir.

23. Resource leak tuzağı

Bu yapı güvenli değildir:

```
var conn = DriverManager.getConnection(url);
var ps = conn.prepareStatement(sql);
var rs = ps.executeQuery(); // burada exception olursa try'a girilmez
try (conn; ps; rs) {
    // use resources
}
```

Son resource oluşturulmadan exception oluşursa try-with-resources statement'ına hiç girilmez; daha önce açılan resource'lar sızabilir. Resource'ları doğrudan try header'ında sırayla oluştur.

Aynı statement üzerinde başka SQL result'u açmak önceki `ResultSet`i kapatabilir. Kapatılmış result üzerinde kullanım `SQLException` oluşturur.

24. SQLException

`SQLException` checked exception'dır; catch edilmeli veya `throws` ile declare edilmelidir.

```
try {
    runQuery();
} catch (SQLException e) {
    System.out.println(e.getMessage());
    System.out.println(e.getSQLState());
    System.out.println(e.getErrorCode());
}
```

Method	Bilgi
<code>getMessage()</code>	Human-readable açıklama
<code>getSQLState()</code>	Standard/vendor SQL state string'i
<code>getErrorCode()</code>	Vendor-specific integer code

Exact message ve error code driver'a bağlıdır; OCP sorusunda verilmedikçe exact metin varsayılmaz.

25. Kaynak resmî cevap ve contract ayrımı

- **Q8 — kaynak A, B:** `{call ...}` portable JDBC escape syntax'idir. Native procedure syntax'i driver'a göre değişebilir.
- **Q14 — kaynak C:** Leading `{?= call ...}` generic procedure `OUT` parameter'ı değil function return value'yu gösterir. Verilen procedure öncülüyle `{call learn(?,?)}` ve ayrı IN/OUT positions gerekir.
- **Q15 — kaynak C, D:** Target savepoint aynı savepoint'e rollback ile otomatik release edilmez. Portable contract'a göre B de mümkündür: **B, C, D**.
- **Q19 — kaynak E:** Plain `UPDATE` metnini `prepareCall()` ile vermek portable değildir; driver prepare veya execute aşamasında reddedebilir.
- **Q21 — kaynak B, D:** B garanti edilir. Line W yokken active transaction `close()` sonucu implementation-defined olduğundan D portable garanti değildir.

26. Derleme, runtime ve output ayrımı

```
var ps = conn.prepareStatement(); // DOES NOT COMPILE
```

No-argument overload yoktur.

```
var ps = conn.prepareStatement("SELECT * FROM exhibits WHERE id=?");
var rs = ps.executeQuery(); // compiles; runtime SQLException
```

Bind parameter set edilmediği için runtime'da driver hatası oluşur.

```
try (var ps = conn.prepareStatement("SELECT id FROM exhibits");
    var rs = ps.executeQuery()) {
    while (rs.next()) {
        System.out.println(rs.getInt(1));
    }
}
```

Derlenir; actual output database içeriğine ve driver'a bağlıdır.

27. Hızlı karar ağacı

```
SQL nerede?
├─ Java application içinde → prepareStatement()
└─ stored procedure → prepareCall()

Sonuç türü?
├─ SELECT rows → executeQuery() → ResultSet
├─ INSERT/UPDATE/DELETE → executeUpdate() → int
└─ önceden bilinmiyor → execute() → boolean

Parameter?
├─ bind/IN → setter
├─ OUT → registerOutParameter()
└─ INOUT → setter + registerOutParameter()

Transaction?
├─ autocommit true → statement sonrası commit
└─ autocommit false → explicit commit()/rollback()
```

28. Özgün mini quiz

Soru 1

Kaç seçenek doğrudur? (İki seçenek seçin.)

```
String sql = "UPDATE exhibits SET num_acres=? WHERE id=?";
try (var ps = conn.prepareStatement(sql)) {
    ps.setDouble(1, 4.5);
    ps.setInt(2, 3);
    int result = ps.executeUpdate();
}
```

- A. Bind index'leri 0'dan başlamalıdır.
- B. Kod uygun `conn` ve exception context'iyle derlenebilir.
- C. `result`, etkilenen row sayısını taşır.
- D. `executeQuery()` kullanılmalıdır.

Soru 2

Bu kodun sınıflandırması nedir?

```
try (var ps = conn.prepareStatement(
    "SELECT name FROM exhibits WHERE id=?");
    var rs = ps.executeQuery()) {
    System.out.println(rs.getString(1));
}
```

- A. Does not compile
- B. Kesin olarak name yazdırır
- C. Runtime'da `SQLException` oluşabilir
- D. Kesin olarak boş çıktı verir

Soru 3

Bir `INOUT` parameter için hangi iki işlem zorunludur?

- A. Yalnız setter
- B. Yalnız `registerOutParameter()`
- C. Setter ve `registerOutParameter()`
- D. `addBatch()` ve `executeBatch()`

Soru 4

Autocommit `false` iken `sp1`, sonra `sp2` oluşturuluyor. Program önce `rollback(sp1)`, sonra `rollback(sp2)` çağırıyor. İkinci çağrı için beklenen sonuç nedir?

- A. Her zaman başarılıdır
- B. `sp2` geçersizleştiği için `SQLException` oluşur
- C. Does not compile
- D. Autocommit otomatik açılır

Soru 5

Try-with-resources header'ında ideal declaration order hangisidir?

- A. `ResultSet`, `PreparedStatement`, `Connection`
- B. `Connection`, `PreparedStatement`, `ResultSet`
- C. Sıra önemsizdir
- D. Yalnız `Connection`

29. Cevaplar ve açıklamalar

- 1 **B, C.** Bind index'i 1'den başlar; update method'u etkilenen row count'ını döndürür.
- 2 **C.** Bind parameter set edilmemiştir; ayrıca cursor `next()` ile valid row'a taşınmamıştır. Kod compile olabilir fakat runtime JDBC hatası üretir.
- 3 **C.** `INOUT`, hem input hem output contract'ını taşır.
- 4 **B.** Daha eski `sp1`e rollback, daha sonra oluşturulan `sp2`yi geçersizleştirir.
- 5 **B.** Reverse close order otomatik olarak `ResultSet` → `statement` → `Connection` olur.

30. Son tekrar

- Beş interface: `Driver`, `Connection`, `PreparedStatement`, `CallableStatement`, `ResultSet`.
- URL: `jdbc:<subprotocol>:<subname>`.
- SQL statement oluşturulurken verilir.
- `executeQuery()` → `ResultSet`; `executeUpdate()` → `int`; `execute()` → result-kind `boolean`.
- Bind parameter ve column index'leri 1'den başlar.
- `ResultSet` getter'ından önce cursor valid row'a taşınır.
- `INOUT` = setter + `registerOutParameter()`.
- Autocommit kapalıysa explicit `commit()` veya `rollback()` gerekir.
- JDBC resource'ları reverse creation order'da kapanır.
- `SQLException` checked exception'dır.